Отчёт по лабораторной работе №1

Web-программирование: «Работа с сокетами в Python»

Выполнил: Клименко Богдан Владимирович, ИСУ №408790, группа K3340
Вариант: 15 (задание 2 – площадь трапеции)
Среда: VS Code + Python

Цель работы

Изучить основы клиент-серверного взаимодействия с использованием сокетов в Python, реализовать различные приложения, работающие по протоколам UDP и TCP, включая многопользовательский чат и простой веб-сервер.

Задание 1: UDP-клиент и сервер (обмен приветствиями)

Протокол: UDP (без установки соединения)

task1_server.py

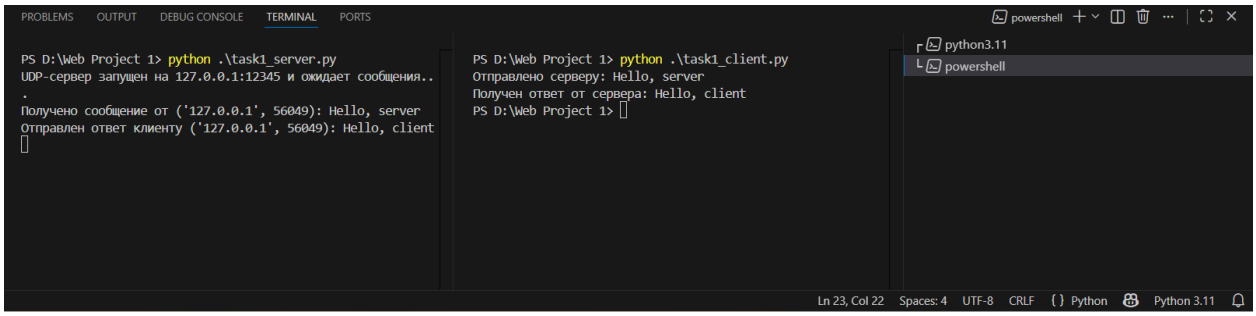
```
import socket

# Настройки сервера
SERVER_HOST = '127.0.0.1'
SERVER_PORT = 12345          # порт для приёма сообщений
BUFFER_SIZE = 1024           # размер буфера для приёма данных

# 1. Создаём UDP-сокет (SOCK_DGRAM)
server_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

# 2. Привязываем сокет к адресу и порту
server_socket.bind((SERVER_HOST, SERVER_PORT))

print(f"UDP-сервер запущен на {SERVER_HOST}:{SERVER_PORT} и ожидает
сообщения...")

try:
    while True:
        # 3. Получаем данные от клиента
        # recvfrom возвращает кортеж: (данные, адрес_клиента)
        data, client_address = server_socket.recvfrom(BUFFER_SIZE)
        message = data.decode('utf-8')
        print(f"Получено сообщение от {client_address}: {message}")

        # 4. Отправляем ответ
        response = "Hello, client"
        server_socket.sendto(response.encode('utf-8'), client_address)
        print(f"Отправлен ответ клиенту {client_address}: {response}")
```

```
except KeyboardInterrupt:
    print("\nСервер остановлен.")
finally:
    server_socket.close()
```

task1_client.py

```
import socket

# Настройки сервера, к которому подключаемся
SERVER_HOST = '127.0.0.1'
SERVER_PORT = 12345
BUFFER_SIZE = 1024

# 1. Создаём UDP-сокет
client_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

# 2. Отправляем сообщение серверу
message = "Hello, server"
client_socket.sendto(message.encode('utf-8'), (SERVER_HOST, SERVER_PORT))
print(f"Отправлено серверу: {message}")

# 3. Получаем ответ от сервера
# recvfrom также возвращает данные и адрес, но адрес нам не обязателен
data, server_address = client_socket.recvfrom(BUFFER_SIZE)
response = data.decode('utf-8')
print(f"Получен ответ от сервера: {response}")

# 4. Закрываем сокет
client_socket.close()
```

Скриншоты работы:



UDP-сокет создаётся с параметром SOCK_DGRAM. Сервер не устанавливает соединение, а просто ждёт датаграммы. Клиент отправляет сообщение и сразу ждёт ответ.

Задание 2: TCP-сервер (вычисление площади трапеции)

Протокол: TCP

Формула: $S = \frac{(a+b)}{2} * h$

task2_server.py

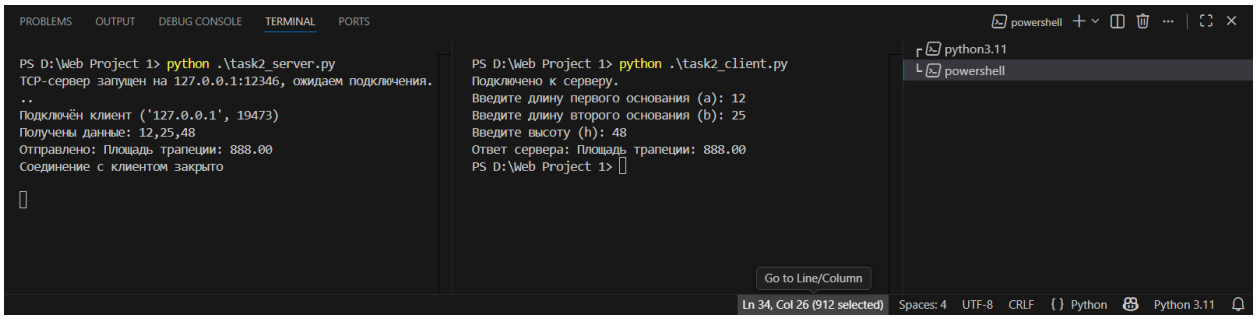
```
import socket


# Настройки сервера
HOST = '127.0.0.1'
PORT = 12346
BUFFER_SIZE = 1024

# Создаём TCP-сокет
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.bind((HOST, PORT))
server_socket.listen(1)
print(f"TCP-сервер запущен на {HOST}:{PORT}, ожидаем подключения...")

try:
    while True:
        # Принимаем подключение
        conn, addr = server_socket.accept()
        print(f"Подключён клиент {addr}")

        # Получаем данные от клиента
        data = conn.recv(BUFFER_SIZE).decode('utf-8')
        print(f"Получены данные: {data}")

        # Разбираем строку вида "a,b,h"
        try:
            a, b, h = map(float, data.split(','))
            if a <= 0 or b <= 0 or h <= 0:
                raise ValueError("Стороны должны быть положительными")
            area = (a + b) / 2 * h
            result = f"Площадь трапеции: {area:.2f}"
        except Exception as e:
            result = f"Ошибка в данных: {e}. Ожидался формат: a,b,h"

        # Отправляем результат клиенту
        conn.send(result.encode('utf-8'))
        print(f"Отправлено: {result}")

        # Закрываем соединение
        conn.close()
        print("Соединение с клиентом закрыто\n")

except KeyboardInterrupt:
    print("\nСервер остановлен.")
```

```
finally:
    server_socket.close()
```

task2_client.py

```
import socket

# Настройки сервера
SERVER_HOST = '127.0.0.1'
SERVER_PORT = 12346
BUFFER_SIZE = 1024

# Создаём TCP-сокет
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

try:
    # Подключаемся к серверу
    client_socket.connect((SERVER_HOST, SERVER_PORT))
    print("Подключено к серверу.")

    # Ввод данных
    a = input("Введите длину первого основания (a): ")
    b = input("Введите длину второго основания (b): ")
    h = input("Введите высоту (h): ")

    # Формируем строку для отправки
    data = f"{a},{b},{h}"
    client_socket.send(data.encode('utf-8'))

    # Получаем ответ
    response = client_socket.recv(BUFFER_SIZE).decode('utf-8')
    print("Ответ сервера:", response)

except ConnectionRefusedError:
    print("Ошибка: сервер не запущен или недоступен.")
except Exception as e:
    print(f"Ошибка: {e}")
finally:
    client_socket.close()
```

Скриншот работы:



TCP обеспечивает надёжное соединение. Сервер принимает подключение, читает

данные, вычисляет результат и отправляет обратно. Клиент подключается, отправляет параметры и получает ответ.

Задание 3: HTTP-сервер с загрузкой index.html

Задача: Отдавать HTML-страницу из файла по запросу браузера.

task3_server.py

```
import socket

# Настройки сервера
HOST = '127.0.0.1'
PORT = 8080
BUFFER_SIZE = 1024

# Создаём TCP-сокет
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)  # избежать
ошибки "Address already in use"
server_socket.bind((HOST, PORT))
server_socket.listen(5)
print(f"HTTP-сервер запущен на http://{HOST}:{PORT}/")

try:
    while True:
        conn, addr = server_socket.accept()
        print(f"Подключение от {addr}")

        # Получаем запрос от клиента (браузера) — можно не использовать, но
прочитать нужно
        request = conn.recv(BUFFER_SIZE).decode('utf-8')
        print(f"Запрос:\n{request.splitlines()[0]}")  # печатаем только первую
строку

        # Пытаемся прочитать файл index.html
        try:
            with open('index.html', 'r', encoding='utf-8') as f:
                html_content = f.read()
            status_line = "HTTP/1.1 200 OK\r\n"
        except FileNotFoundError:
            html_content = "<h1>404 Not Found</h1><p>Файл index.html не
найден.</p>"
            status_line = "HTTP/1.1 404 Not Found\r\n"

        # Формируем HTTP-ответ
        headers = (
            f"Content-Type: text/html; charset=utf-8\r\n"
            f"Content-Length: {len(html_content.encode('utf-8'))}\r\n"
            "Connection: close\r\n"
            "\r\n"
```

```
        )
        http_response = status_line + headers + html_content

        # Отправляем ответ
        conn.sendall(http_response.encode('utf-8'))
        conn.close()
        print("Ответ отправлен, соединение закрыто\n")

except KeyboardInterrupt:
    print("\nСервер остановлен.")
finally:
    server_socket.close()
```

Содержимое index.html

```
<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <title>Простой сервер</title>
</head>
<body>
    <h1>Хелоу ворлд!</h1>
    <p>Страница index.html, загруженная через сокеты.</p>
</body>
</html>
```

Скриншот работы:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS                                   python3.11

PS D:\Web Project 1> python .\task3_server.py
HTTP-сервер запущен на http://127.0.0.1:8080/
Подключение от ('127.0.0.1', 49738)
Запрос:
GET / HTTP/1.1
Ответ отправлен, соединение закрыто

Подключение от ('127.0.0.1', 49739)
Запрос:
GET /favicon.ico HTTP/1.1
Ответ отправлен, соединение закрыто

Ln 7, Col 7   Spaces: 4   UTF-8   CRLF   {} HTML
```

Сервер читает файл index.html, формирует корректный HTTP-ответ с заголовками и отправляет клиенту (браузеру). После каждого запроса соединение закрывается.

Задание 4: Многопользовательский чат (TCP + threading)

Протокол: TCP, многопоточность

chat_server.py

```python
import socket
import threading

# Настройки сервера
HOST = '127.0.0.1'
PORT = 12347

# Список подключенных клиентов: каждый элемент - кортеж (сокет, имя)
clients = []

def broadcast(message, sender_socket=None):
    """
    Отправляет сообщение всем клиентам, кроме отправителя.
    Если sender_socket = None, сообщение отправляется всем.
    """
    for client_socket, _ in clients:
        if client_socket != sender_socket:
            try:
                client_socket.send(message.encode('utf-8'))
            except:
                # Если клиент отключился, удалим его при следующей итерации
                pass

def handle_client(client_socket, addr):
    """
    Функция для обработки одного клиента в отдельном потоке.
    """
    # Запрашиваем имя у клиента
    try:
        client_socket.send("Введите ваше имя: ".encode('utf-8'))
        name = client_socket.recv(1024).decode('utf-8').strip()
        if not name:
```

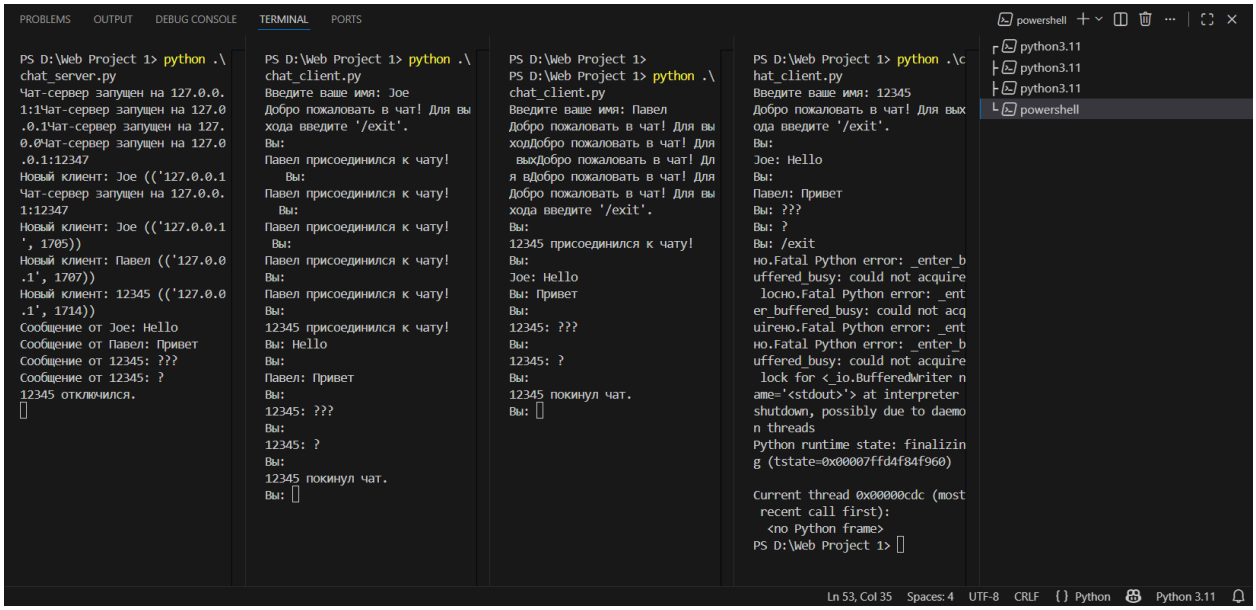
```
            name = f"User{addr[1]}"  # если имя пустое, генерируем
    except:
        # Если не удалось получить имя, закрываем соединение
        client_socket.close()
        return

    # Добавляем клиента в список
    clients.append((client_socket, name))
    print(f"Новый клиент: {name} ({addr})")
    broadcast(f"{name} присоединился к чату!", client_socket)

    try:
        while True:
            # Получаем сообщение от клиента
            message = client_socket.recv(1024).decode('utf-8')
            if not message:
                # Если recv вернул пустую строку, клиент отключился
                break
            print(f"Сообщение от {name}: {message}")
            broadcast(f"{name}: {message}", client_socket)
    except:
        # Любая ошибка (например, разрыв соединения) приводит к выходу из цикла
        pass
    finally:
        # Удаляем клиента из списка
        clients[:] = [c for c in clients if c[0] != client_socket]
        client_socket.close()
        print(f"{name} отключился.")
        broadcast(f"{name} покинул чат.", None)

# Создаём серверный сокет
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)  # чтобы
быстро переиспользовать порт
server_socket.bind((HOST, PORT))
server_socket.listen(5)
print(f"Чат-сервер запущен на {HOST}:{PORT}")

try:
    while True:
        client_socket, addr = server_socket.accept()
        # Создаём поток для нового клиента
        thread = threading.Thread(target=handle_client, args=(client_socket,
addr))
        thread.daemon = True  # поток завершится при завершении основного потока
        thread.start()
except KeyboardInterrupt:
    print("\nСервер остановлен.")
finally:
    server_socket.close()
```

chat_client.py

```
import socket
import threading
import sys

SERVER_HOST = '127.0.0.1'
SERVER_PORT = 12347

def receive_messages(sock):
    """Поток для получения сообщений от сервера (запускается после ввода
имени)."""
    try:
        while True:
            message = sock.recv(1024).decode('utf-8')
            if not message:
                break
            # Выводим сообщение и затем приглашение для ввода
            sys.stdout.write(f"\n{message}\n")
            sys.stdout.write("Вы: ")
            sys.stdout.flush()
    except:
        pass
    finally:
        print("\nСоединение с сервером потеряно.")
        sock.close()

# Подключаемся к серверу
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
try:
    client.connect((SERVER_HOST, SERVER_PORT))
except ConnectionRefusedError:
    print("Ошибка: сервер не запущен. Запустите сервер и повторите попытку.")
    sys.exit(1)
try:
    prompt = client.recv(1024).decode('utf-8')
    print(prompt, end="")          # выводим "Введите ваше имя: "
    name = input()                  # вводим имя
    client.send(name.encode('utf-8'))
except:
    print("Ошибка при вводе имени.")
    client.close()
    sys.exit(1)

# Теперь запускаем поток для получения сообщений
recv_thread = threading.Thread(target=receive_messages, args=(client,))
recv_thread.daemon = True
recv_thread.start()

print("Добро пожаловать в чат! Для выхода введите '/exit'.")
```

```python
# Основной цикл отправки сообщений
while True:
    try:
        msg = input("Вы: ")
        if msg.lower() == '/exit':
            break
        client.send(msg.encode('utf-8'))
    except KeyboardInterrupt:
        break
    except:
        print("Ошибка отправки.")
        break

client.close()
```

Скриншот работы:



Сервер создаёт отдельный поток для каждого клиента. Потоки слушают входящие сообщения и рассылают их всем остальным. Клиент также использует два потока: один для ввода, другой для получения сообщений, чтобы не блокировать интерфейс.

Задание 5: Веб-сервер для хранения оценок (GET/POST, JSON)

Протокол: HTTP
Хранение: JSON-файл

grade_server.py

```python
import socket
import urllib.parse
import json
import os
```

```python
# ================== НАСТРОЙКИ ==================
HOST = '127.0.0.1'
PORT = 8081
GRADES_FILE = 'grades.json'

# ================== РАБОТА С ФАЙЛОМ ==================
def load_grades():
    if os.path.exists(GRADES_FILE):
        with open(GRADES_FILE, 'r', encoding='utf-8') as f:
            return json.load(f)
    return []

def save_grades(grades):
    with open(GRADES_FILE, 'w', encoding='utf-8') as f:
        json.dump(grades, f, ensure_ascii=False, indent=2)

grades = load_grades()

# ================== HTML-ШАБЛОН ==================
HTML_TEMPLATE = """<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <title>Оценки по дисциплинам</title>
    <style>
        table {{ border-collapse: collapse; width: 50%; }}
        th, td {{ border: 1px solid black; padding: 8px; text-align: left; }}
        th {{ background-color: #f2f2f2; }}
        form {{ margin-top: 20px; }}
    </style>
</head>
<body>
    <h1>Список оценок</h1>
    <table>
        <tr><th>Дисциплина</th><th>Оценка</th></tr>
        {rows}
    </table>
    <h2>Добавить оценку</h2>
    <form method="POST" action="/">
        Дисциплина: <input type="text" name="discipline" required><br>
        Оценка: <input type="number" name="grade" min="1" max="5" required><br>
        <input type="submit" value="Отправить">
    </form>
</body>
</html>
"""

# ================== ЧТЕНИЕ ЗАПРОСА ПОЛНОСТЬЮ ==================
def recv_all(conn):
    """
```

```
    Читает все данные из сокета до тех пор, пока не будет получен полный HTTP-
запрос.
    Возвращает байтовую строку запроса.
    """
    data = b''
    while True:
        try:
            chunk = conn.recv(4096)
            if not chunk:
                break
            data += chunk
            # Проверяем, получили ли мы конец заголовков (двойной перевод строки)
            if b'\r\n\r\n' in data:
                # Заголовки получены, теперь нужно определить, есть ли тело и
сколько байт читать
                headers_end = data.find(b'\r\n\r\n') + 4
                headers = data[:headers_end].decode('utf-8', errors='ignore')
                # Ищем Content-Length
                content_length = 0
                for line in headers.split('\r\n'):
                    if line.lower().startswith('content-length:'):
                        content_length = int(line.split(':')[1].strip())
                        break
                # Если тело ожидается и мы уже получили его полностью - выходим
                if len(data) - headers_end >= content_length:
                    break
                # Иначе продолжаем читать, пока не получим нужное количество байт
        except socket.timeout:
            # Таймаут не устанавливаем, чтобы избежать ошибок
            break
    return data

# ================= ПАРСИНГ POST-ДАННЫХ =================
def parse_post_data(data):
    """Разбирает строку вида 'discipline=Math&grade=5' в словарь."""
    params = urllib.parse.parse_qs(data)
    discipline = params.get('discipline', [''])[0]
    grade = params.get('grade', [''])[0]
    return discipline, grade

# ================= ОБРАБОТКА ЗАПРОСА =================
def handle_request(request_data):
    """
    Обрабатывает HTTP-запрос и возвращает кортеж (статус, тело_ответа,
доп_заголовки)
    """
    # Разбиваем запрос на строки (первая строка - request line)
    # Сначала декодируем только заголовочную часть для разбора
    # Но проще работать с байтами, но для простоты оставим строку
    # Запрос уже в виде строки, но нужно разделить заголовки и тело
    try:
```

```
        # Ищем разделитель между заголовками и телом
        header_end = request_data.find('\r\n\r\n')
        if header_end == -1:
            return "400 Bad Request", "<h1>400 Bad Request (No headers
end)</h1>", []

        headers_part = request_data[:header_end]
        body_part = request_data[header_end+4:]

        lines = headers_part.split('\r\n')
        if not lines:
            return "400 Bad Request", "<h1>400 Bad Request</h1>", []

        request_line = lines[0]
        parts = request_line.split()
        if len(parts) < 2:
            return "400 Bad Request", "<h1>400 Bad Request</h1>", []

        method, path = parts[0], parts[1]

        # Игнорируем запросы favicon.ico
        if path == '/favicon.ico':
            return "204 No Content", "", []

        if path != '/':
            return "404 Not Found", "<h1>404 Not Found</h1>", []

        # --- GET запрос ---
        if method == 'GET':
            rows = ''
            for disc, grade in grades:
                rows += f"<tr><td>{disc}</td><td>{grade}</td></tr>\n"
            body = HTML_TEMPLATE.format(rows=rows)
            return "200 OK", body, []

        # --- POST запрос ---
        elif method == 'POST':
            # Извлекаем Content-Length из заголовков
            content_length = 0
            for line in lines:
                if line.lower().startswith('content-length:'):
                    content_length = int(line.split(':')[1].strip())
                    break

            if content_length == 0:
                return "400 Bad Request", "<h1>400 Bad Request (No Content-
Length)</h1>", []

            # Тело должно быть уже в body_part, но если его нет (редко), пробуем
прочитать ещё
            if len(body_part) < content_length:
```

```
                # В реальности мы уже должны были получить всё в recv_all, но на
всякий случай
                return "400 Bad Request", "<h1>400 Bad Request (Incomplete
body)</h1>", []

            post_data = body_part[:content_length]
            discipline, grade = parse_post_data(post_data)

            if not discipline or not grade:
                return "400 Bad Request", "<h1>Не хватает данных (discipline и
grade обязательны)</h1>", []

            try:
                grade = int(grade)
                grades.append((discipline, grade))
                save_grades(grades)
                print(f"Добавлено: {discipline} - {grade}")
                return "303 See Other", "", [("Location", "/")]
            except ValueError:
                return "400 Bad Request", "<h1>Оценка должна быть числом</h1>",
[]

        else:
            return "405 Method Not Allowed", "<h1>405 Method Not Allowed</h1>",
[]

    except Exception as e:
        # Если что-то пошло не так, возвращаем 500 с текстом ошибки для отладки
        return "500 Internal Server Error", f"<h1>500 Internal Server
Error</h1><p>{e}</p>", []

# ================= ЗАПУСК СЕРВЕРА =================
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
server_socket.bind((HOST, PORT))
server_socket.listen(5)
print(f"Веб-сервер запущен на http://{HOST}:{PORT}/")

try:
    while True:
        conn, addr = server_socket.accept()
        print(f"Подключение от {addr}")

        # Устанавливаем небольшой таймаут для предотвращения зависаний, но чтение
теперь умное
        conn.settimeout(5.0)

        try:
            # Читаем запрос полностью с помощью recv_all
            raw_request = recv_all(conn)
            if not raw_request:
```

```
                conn.close()
                continue

            # Декодируем запрос в строку (ошибки игнорируем)
            request_data = raw_request.decode('utf-8', errors='ignore')

            # Обрабатываем запрос
            status, body, extra_headers = handle_request(request_data)

        except socket.timeout:
            status, body, extra_headers = "408 Request Timeout", "<h1>408 Request
Timeout</h1>", []
        except Exception as e:
            status, body, extra_headers = "500 Internal Server Error", f"<h1>500
Internal Server Error</h1><p>{e}</p>", []

        # Формируем ответ
        response_line = f"HTTP/1.1 {status}\r\n"
        headers = [
            f"Content-Length: {len(body.encode('utf-8'))}",
            "Content-Type: text/html; charset=utf-8",
            "Connection: close"
        ]
        for header_name, header_value in extra_headers:
            headers.append(f"{header_name}: {header_value}")

        response = response_line + "\r\n".join(headers) + "\r\n\r\n" + body

        # Отправляем ответ
        conn.sendall(response.encode('utf-8'))
        conn.close()
        print(f"Ответ отправлен со статусом {status}")

except KeyboardInterrupt:
    print("\nСервер остановлен.")
finally:
    server_socket.close()
```

Скриншот работы:

127.0.0.1:8081

Список оценок

Дисциплина	Оценка
Математика	4
Русский язык	2
Матанализ	3
Обществознание	5
Информатика	5
22	2

Добавить оценку

Дисциплина: Web-программирование
Оценка: 5
Отправить

127.0.0.1:8081

Список оценок

Дисциплина	Оценка
Математика	4
Русский язык	2
Матанализ	3
Обществознание	5
Информатика	5
22	2
Web-программирование	5

Добавить оценку

Дисциплина:
Оценка:
Отправить

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS python3.11

```
PS D:\Web Project 1> python.exe .\grade_server.py
Веб-сервер запущен на http://127.0.0.1:8081/
Подключение от ('127.0.0.1', 57843)
Подключение от ('127.0.0.1', 57844)
Ответ отправлен со статусом 200 OK
Подключение от ('127.0.0.1', 57853)
Подключение от ('127.0.0.1', 57854)
Добавлено: Web-программирование - 5
Ответ отправлен со статусом 303 See Other
Подключение от ('127.0.0.1', 57855)
Ответ отправлен со статусом 200 OK
```

Ln 230, Col 26 Spaces: 4 UTF-8 CRLF { } Python Python 3.11

Содержание **grades.json**:

```
[
  [
    "Математика",
    4
  ],
  [
```

```
    "Русский язык",
    2
  ],
  [
    "Матанализ",
    3
  ],
  [
    "Обществознание",
    5
  ],
  [
    "Информатика",
    5
  ],
  [
    "22",
    2
  ],
  [
    "Web-программирование",
    5
  ]
]
```


Сервер разбирает метод и путь запроса. Для GET генерирует HTML-таблицу из списка оценок. Для POST извлекает данные формы, добавляет запись в список и сохраняет в JSON. После добавления выполняет редирект, чтобы избежать повторной отправки формы.

Выводы

В ходе лабораторной работы я:

- Изучил основные протоколы транспортного уровня (UDP и TCP).
- Научился создавать клиент-серверные приложения с использованием библиотеки socket.
- Реализовал многопользовательский чат с применением многопоточности.
- Разработал полноценный веб-сервер, обрабатывающий GET и POST запросы и сохраняющий данные в JSON-файл.
- Получил практические навыки отладки сетевых приложений в Windows.